

20 ĐỀ KIỂM TRA PYTHON CB-NC

Thời gian: 90 phút - Chỉ được dùng tài liệu giấy

Mỗi câu 2 điểm

ĐỀ KIỂM TRA PYTHON CƠ BẢN – CB01

Thời gian làm bài: 90 phút

Bài 1: Tính toán số học, logic

- a) Nhập vào hai số nguyên a, b . In ra giá trị $a^2 + b^2$.
- b) Nhập vào một số nguyên n . In ra **True** nếu n là số chẵn, ngược lại in **False**.

Bài 2: Xử lý dữ liệu (list, dict, set)

- a) Nhập một dãy số nguyên (trên một dòng, cách nhau bởi dấu cách), in ra số lớn thứ hai (không trùng).
- b) Nhập một chuỗi, in ra từ điển tần suất ký tự trong chuỗi.

Bài 3: Hàm tính toán thuần (pure function)

- a) Viết hàm `factorial(n)` trả về giai thừa của n . Nhập n từ bàn phím, gọi hàm và in kết quả.
- b) Viết hàm `is_prime(n)` trả về **True** nếu n là số nguyên tố, ngược lại **False**. Nhập n và in kết quả.

Bài 4: Xử lý chuỗi, số lượng ký tự, đảo chuỗi

- a) Nhập một chuỗi, in ra chuỗi đảo ngược.
- b) Nhập một chuỗi, in ra số lượng ký tự khác nhau trong chuỗi.

Bài 5: Lập trình hướng đối tượng với đầu ra ổn định

- a) Xây dựng lớp `SinhVien` với thuộc tính `ten`, `mssv`. Khởi tạo một đối tượng với tên “Nguyen Van A”, MSSV “123456”, gọi phương thức `in_thong_tin()` để in thông tin theo định dạng “Nguyen Van A - 123456”.
- b) Thêm thuộc tính `diem` vào lớp `SinhVien`. Khởi tạo đối tượng với tên “Le Thi B”, MSSV “234567”, điểm 8.5, gọi phương thức `in_thong_tin()` để in thông tin theo định dạng “Le Thi B - 234567 - 8.5”.

ĐỀ KIỂM TRA PYTHON CƠ BẢN – CB02

Thời gian làm bài: 90 phút

Bài 1: Tính toán số học, logic

- a) Nhập vào hai số nguyên x, y . In ra tổng và hiệu của chúng.
- b) Nhập vào số nguyên n . In ra `True` nếu n là số lẻ, ngược lại in `False`.

Bài 2: Xử lý dữ liệu (list, dict, set)

- a) Nhập một dãy số nguyên, in ra số nhỏ nhất.
- b) Nhập một chuỗi, in ra số lượng ký tự xuất hiện đúng một lần trong chuỗi.

Bài 3: Hàm tính toán thuần (pure function)

- a) Viết hàm `tong_so_chan(n)` trả về tổng các số chẵn từ 1 đến n . Nhập n từ bàn phím, gọi hàm và in ra kết quả.
- b) Viết hàm `is_perfect(n)` trả về `True` nếu n là số hoàn hảo, ngược lại in `False`. Nhập n và in kết quả.

Bài 4: Xử lý chuỗi, số lượng ký tự, đảo chuỗi

- a) Nhập một chuỗi, in ra chuỗi viết hoa toàn bộ.
- b) Nhập một chuỗi, in ra chuỗi chỉ giữ lại các ký tự chữ số (0–9).

Bài 5: Lập trình hướng đối tượng với đầu ra ổn định

- a) Xây dựng lớp `Sach` với thuộc tính `ten_sach`, `tac_gia`. Khởi tạo một đối tượng với tên “Python Co Ban”, tác giả “Tran Van C”, gọi phương thức `in_thong_tin()` để in ra “Python Co Ban - Tran Van C”.
- b) Thêm thuộc tính `nam_xuat_ban` vào lớp `Sach`. Khởi tạo đối tượng với tên “Lap Trinh Nang Cao”, tác giả “Nguyen Van D”, năm xuất bản 2024, gọi phương thức `in_thong_tin()` để in ra “Lap Trinh Nang Cao - Nguyen Van D - 2024”.

ĐỀ KIỂM TRA PYTHON CƠ BẢN – CB03

Thời gian làm bài: 90 phút

Bài 1: Tính toán số học, logic

- a) Nhập vào hai số thực x, y . In ra giá trị $|x - y|$ (giá trị tuyệt đối).
- b) Nhập vào số nguyên n . In ra “Yes” nếu n chia hết cho 5, ngược lại in “No”.

Bài 2: Xử lý dữ liệu (list, dict, set)

- a) Nhập một dãy số nguyên, in ra tập hợp các số khác nhau (dạng set Python).
- b) Nhập một chuỗi, in ra tổng số ký tự là chữ cái (A-Z, a-z).

Bài 3: Hàm tính toán thuần (pure function)

- a) Viết hàm `tong_le(n)` trả về tổng các số lẻ từ 1 đến n . Nhập n từ bàn phím, gọi hàm và in ra kết quả.
- b) Viết hàm `so_dao_nguoc(n)` trả về số đảo ngược của n . Nhập n và in kết quả.

Bài 4: Xử lý chuỗi, số lượng ký tự, đảo chuỗi

- a) Nhập một chuỗi, in ra chuỗi sau khi loại bỏ các khoảng trắng.
- b) Nhập một chuỗi, in ra chuỗi với từng ký tự cách nhau bởi dấu ‘-’.

Bài 5: Lập trình hướng đối tượng với đầu ra ổn định

- a) Xây dựng lớp `HinhTron` với thuộc tính `ban_kinh`. Khởi tạo đối tượng với bán kính 2.5, gọi phương thức `chu_vi()` để in ra chu vi (làm tròn 2 chữ số thập phân, công thức $2\pi r$ với $\pi = 3.14$).
- b) Thêm phương thức `dien_tich()` vào lớp `HinhTron`. Khởi tạo đối tượng với bán kính 1.2, in ra diện tích làm tròn 2 chữ số (công thức πr^2 với $\pi = 3.14$).

ĐỀ KIỂM TRA PYTHON CƠ BẢN – CB04

Thời gian làm bài: 90 phút

Bài 1: Tính toán số học, logic

- a) Nhập vào hai số thực a, b . In ra tích của chúng.
- b) Nhập vào một số nguyên n . In ra **True** nếu n là số âm, ngược lại in **False**.

Bài 2: Xử lý dữ liệu (list, dict, set)

- a) Nhập một dãy số nguyên, in ra danh sách các số chẵn theo thứ tự tăng dần.
- b) Nhập một chuỗi, in ra ký tự xuất hiện nhiều nhất trong chuỗi.

Bài 3: Hàm tính toán thuần (pure function)

- a) Viết hàm `tong_cac_uoc(n)` trả về tổng các ước của n (không kể chính nó). Nhập n , gọi hàm và in ra kết quả.
- b) Viết hàm `is_square(n)` trả về **True** nếu n là số chính phương, ngược lại in **False**. Nhập n và in kết quả.

Bài 4: Xử lý chuỗi, số lượng ký tự, đảo chuỗi

- a) Nhập một chuỗi, in ra chuỗi chỉ gồm các ký tự in hoa.
- b) Nhập một chuỗi, in ra số lần xuất hiện của ký tự đầu tiên trong chuỗi.

Bài 5: Lập trình hướng đối tượng với đầu ra ổn định

- a) Xây dựng lớp `HinhVuong` với thuộc tính `cạnh`. Khởi tạo đối tượng với cạnh 3.0, gọi phương thức `chu_vi()` để in ra chu vi (4 nhân cạnh).
- b) Thêm phương thức `dien_tich()` vào lớp `HinhVuong`. Khởi tạo đối tượng với cạnh 2.5, in ra diện tích (cạnh bình phương).

ĐỀ KIỂM TRA PYTHON CƠ BẢN – CB05

Thời gian làm bài: 90 phút

Bài 1: Tính toán số học, logic

- a) Nhập vào hai số thực a, b . In ra thương (chia thực) của a cho b .
- b) Nhập vào số nguyên n . In ra “Yes” nếu n là số dương, ngược lại in “No”.

Bài 2: Xử lý dữ liệu (list, dict, set)

- a) Nhập một dãy số nguyên, in ra tổng các số lẻ trong dãy.
- b) Nhập một chuỗi, in ra các ký tự không lặp lại (xuất hiện đúng một lần).

Bài 3: Hàm tính toán thuần (pure function)

- a) Viết hàm `tim_max(lst)` trả về số lớn nhất trong danh sách `lst`. Nhập dãy số, gọi hàm và in ra kết quả.
- b) Viết hàm `dem_uoc(n)` trả về số lượng ước số của n . Nhập n , gọi hàm và in ra kết quả.

Bài 4: Xử lý chuỗi, số lượng ký tự, đảo chuỗi

- a) Nhập một chuỗi, in ra chuỗi sau khi loại bỏ các ký tự số.
- b) Nhập một chuỗi, in ra chuỗi chỉ giữ lại các ký tự chữ thường.

Bài 5: Lập trình hướng đối tượng với đầu ra ổn định

- a) Xây dựng lớp `MayTinh` với thuộc tính `ten_may`, `gia`. Khởi tạo đối tượng với tên “Dell XPS”, giá 35000, gọi phương thức `in_thong_tin()` để in “Dell XPS - 35000”.
- b) Thêm thuộc tính `nam_sx` vào lớp `MayTinh`. Khởi tạo đối tượng với tên “MacBook Air”, giá 42000, năm sản xuất 2023, gọi phương thức `in_thong_tin()` để in “MacBook Air - 42000 - 2023”.

ĐỀ KIỂM TRA PYTHON CƠ BẢN – CB06

Thời gian làm bài: 90 phút

Câu 1 (2 điểm) – Số học và xử lý số (Chương 1, 2)

Nhập số nguyên dương n (từ 10 đến 1000). Viết chương trình:

- In các chữ số của n theo chiều ngược lại.
- Tính tổng các chữ số lẻ trong n .
- Kiểm tra xem n có phải số Armstrong không (tổng lập phương các chữ số bằng chính số đó, ví dụ: 153).

Câu 2 (2 điểm) – Xử lý danh sách (Chương 2)

Nhập vào danh sách các số nguyên (cách nhau bởi dấu cách). Viết chương trình:

- In ra số nguyên âm lớn nhất (nếu không có, in thông báo phù hợp).
- Đếm và in ra số lượng số nguyên dương lẻ.
- Đảo ngược danh sách và in ra.

Câu 3 (2 điểm) – Thao tác chuỗi nâng cao (Chương 2)

Nhập vào một chuỗi bất kỳ. Viết chương trình:

- Đếm số lần xuất hiện của từng nguyên âm (a,e,i,o,u).
- Xoá tất cả nguyên âm ra khỏi chuỗi.
- In chuỗi kết quả sau khi xoá nguyên âm.

Câu 4 (2 điểm) – File và xử lý file văn bản (Chương 2)

Giả sử có file `text.txt` chứa một đoạn văn bản. Viết chương trình:

- Đọc file, đếm tổng số từ trong file.
- Tìm và ghi vào file `longest.txt` từ dài nhất trong file. Nếu có nhiều từ dài nhất bằng nhau, ghi tất cả mỗi từ một dòng.

Câu 5 (2 điểm) – Lập trình hướng đối tượng (Chương 3, 4)

Xây dựng lớp `SanPham` có các thuộc tính: `ten` (str), `so_luong` (int), `don_gia` (float). Viết phương thức `thanh_tien()` trả về giá trị số lượng nhân với đơn giá.

Nhập thông tin 2 sản phẩm, in ra sản phẩm có thành tiền lớn hơn.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC07

Thời gian làm bài: 90 phút

Câu 1 (2 điểm): Nhập 2 số nguyên dương a, b ($1 \leq a < b \leq 1000$). Viết chương trình:

- a) Tính tổng các số chính phương trong đoạn $[a, b]$.
- b) Viết hàm trả về ước chung lớn nhất (UCLN) và bội chung nhỏ nhất (BCNN) của a và b .

Câu 2 (2 điểm): Viết chương trình nhập chuỗi từ bàn phím và thực hiện:

- a) Đếm số ký tự đặc biệt (không phải chữ cái, không phải chữ số).
- b) Đảo ngược thứ tự các từ trong chuỗi.
- c) Kiểm tra chuỗi nhập vào có phải là palindrome không (không tính dấu cách, hoa thường).

Câu 3 (2 điểm): Viết chương trình nhập danh sách số nguyên, thực hiện:

- a) Xoá hết các số nguyên tố khỏi danh sách.
- b) Thêm vào cuối danh sách số lớn nhất ban đầu.
- c) In danh sách đã sắp xếp giảm dần ra màn hình.

Câu 4 (2 điểm): Cho file "numbers.txt" lưu nhiều số nguyên trên một dòng cách nhau bởi dấu cách. Viết chương trình:

- a) Đọc file, tìm các số hoàn hảo, ghi vào file "perfect.txt".
- b) Nếu không tìm thấy, ghi "Khong tim thay" vào file "perfect.txt".
- c) Thêm vào cuối file "numbers.txt" dòng "Ho ten sinh vien: [HoTen]".

Câu 5 (2 điểm – OOP nâng cao): Xây dựng lớp MayTinh với thuộc tính: ten, gia, cpu. Viết lớp kế thừa Laptop bổ sung thêm thuộc tính: trong_luong và phương thức in thông tin chi tiết máy. Tạo hai đối tượng Laptop với thông tin cụ thể và gọi phương thức in thông tin của chúng.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC08

Thời gian làm bài: 90 phút

Câu 1 (2 điểm) – Đệ quy và Fibonacci

Viết hàm đệ quy tính số Fibonacci thứ n (biết $\text{Fibonacci}(0)=0$, $\text{Fibonacci}(1)=1$). In dãy Fibonacci gồm n số đầu tiên, với n được nhập vào từ bàn phím.

Câu 2 (2 điểm) – Xử lý danh sách

Nhập vào một dãy các số nguyên. Viết chương trình thực hiện:

- Sắp xếp danh sách tăng dần bằng thuật toán sắp xếp nổi bọt (bubble sort).
- Tìm phần tử xuất hiện nhiều nhất trong danh sách.
- In ra các phần tử duy nhất trong danh sách theo thứ tự ban đầu.

Câu 3 (2 điểm) – Dictionary và tập hợp (Set)

Nhập vào hai chuỗi ký tự. Viết chương trình:

- In ra các ký tự chung của hai chuỗi.
- In ra các ký tự chỉ xuất hiện trong chuỗi thứ nhất nhưng không có trong chuỗi thứ hai.
- Tạo dictionary chứa các ký tự và tổng số lần xuất hiện của mỗi ký tự trong cả hai chuỗi.

Câu 4 (2 điểm) – Thao tác File JSON

Cho file `data.json` chứa danh sách các từ điển dạng:
`[{"ten": "An", "tuoi": 22, ...}]`. Viết chương trình:

- Đọc file và in tên người có tuổi lớn nhất.
- Tính và in tuổi trung bình của tất cả mọi người.

Câu 5 (2 điểm) – OOP nâng cao

Viết lớp `HinhTron` có thuộc tính bán kính `r`. Viết phương thức tính diện tích và chu vi. Viết lớp con `HinhTru` kế thừa từ lớp `HinhTron`, bổ sung thuộc tính `chieu_cao` và phương thức tính thể tích hình trụ.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC09

Thời gian làm bài: 90 phút

Câu 1 (2 điểm): Nhập vào hai số nguyên dương m, n ($1 \leq m < n \leq 500$). Viết chương trình thực hiện:

- a) In ra các số nguyên tố nằm trong đoạn $[m, n]$.
- b) Viết hàm trả về tổng các chữ số của số nguyên n .

Câu 2 (2 điểm): Viết chương trình nhập chuỗi ký tự từ bàn phím, thực hiện:

- a) Đếm số từ có chứa ký tự số.
- b) Thay thế tất cả các số trong chuỗi bằng dấu $*$.
- c) In ra chuỗi kết quả.

Câu 3 (2 điểm): Nhập vào danh sách số nguyên, viết chương trình:

- a) Tạo hai danh sách mới, một chứa số lẻ và một chứa số chẵn từ danh sách ban đầu.
- b) Xóa các phần tử trùng nhau trong mỗi danh sách.
- c) In cả hai danh sách kết quả theo thứ tự tăng dần.

Câu 4 (2 điểm): Cho file văn bản `input.txt` gồm nhiều dòng. Viết chương trình:

- a) Đọc file và đếm tổng số dòng.
- b) Ghi các dòng chứa từ "Python" vào file `python_lines.txt`.
- c) Thêm vào cuối file `input.txt` dòng: "MSSV: [MSSV của bạn]".

Câu 5 (2 điểm – OOP có kế thừa): Xây dựng lớp `NhanVien` có các thuộc tính: `ten`, `luong_cb`. Viết lớp con `NhanVienBanHang` kế thừa thêm thuộc tính `doanh_thu` và phương thức tính lương tổng (lương cơ bản + 5% doanh thu). Tạo đối tượng `NhanVienBanHang`, in thông tin và tổng lương.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC10

Thời gian làm bài: 90 phút

Câu 1 (2 điểm) – Độ quy và xử lý số

Viết hàm đệ quy:

- a) Tính tổng các chữ số của một số nguyên dương.
- b) Tìm ước chung lớn nhất (UCLN) của hai số nguyên dương.

Câu 2 (2 điểm) – List comprehension

Cho danh sách số nguyên. Viết chương trình dùng list comprehension để:

- a) Tạo danh sách bình phương các số lẻ.
- b) Tạo danh sách các số nguyên tố.
- c) Tạo danh sách các tuple dạng (số, bình phương của số đó).

Câu 3 (2 điểm) – Dictionary nâng cao

Viết chương trình đọc vào một đoạn văn bản, thực hiện:

- a) Tạo từ điển đếm tần suất xuất hiện của từng từ.
- b) In ra 3 từ xuất hiện nhiều nhất.
- c) In ra từ có độ dài dài nhất (nếu có nhiều từ cùng độ dài, chọn từ xuất hiện đầu tiên).

Câu 4 (2 điểm) – Xử lý file CSV

Giả sử có file `diem.csv` chứa các hàng: `Ten,HocPhan,Diem`. Viết chương trình:

- a) Đọc dữ liệu và in ra điểm trung bình theo từng học phần.
- b) Tìm sinh viên có điểm cao nhất mỗi học phần.
- c) Ghi kết quả mục b vào file `top.csv`.

Câu 5 (2 điểm) – OOP và kế thừa

Tạo lớp cơ sở `Ngnoi` (thuộc tính: `ten`, `tuoi`), lớp con `SinhVien` kế thừa từ lớp `Ngnoi` (thêm `diem_tb`). Viết hàm xuất thông tin và xếp loại sinh viên (`Gioi` nếu `diem_tb` ≥ 8 , `Kha` nếu `diem_tb` ≥ 6.5 , còn lại là `Trung Binh`).

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC11

Thời gian làm bài: 90 phút

Câu 1 (2 điểm) – NumPy cơ bản

Viết chương trình sử dụng NumPy:

- a) Tạo mảng 20 số nguyên ngẫu nhiên từ 10 đến 50.
- b) Tính trung vị (median) và tổng các số chẵn trong mảng.

Câu 2 (2 điểm) – NumPy nâng cao

Tạo một ma trận ngẫu nhiên 4×4 với các phần tử từ 1-20:

- a) In ra ma trận và chuyển vị của nó.
- b) In ra eigenvalues (trị riêng) của ma trận.

Câu 3 (2 điểm) – NumPy thống kê

Tạo mảng NumPy với 200 số thực ngẫu nhiên theo phân phối đều từ 0 đến 1:

- a) Tính phần trăm các số nhỏ hơn 0.5 trong mảng.
- b) Tính giá trị tứ phân vị (quartiles) của mảng.

Câu 4 (2 điểm) – Pandas cơ bản

Cho file `data.csv` chứa dữ liệu về điểm các môn học của sinh viên dạng: `Ten,HocPhan,Diem`. Dùng pandas thực hiện:

- a) Đọc dữ liệu vào DataFrame và in ra thông tin thống kê cơ bản.
- b) In danh sách sinh viên có điểm trên 8 ở môn "Python".
- c) Tính và in ra điểm trung bình của mỗi môn học.

Câu 5 (2 điểm) – OOP nâng cao (kế thừa)

Viết lớp cơ sở `NhanVien` gồm thuộc tính: tên, lương cơ bản. Viết lớp con `QuanLy` kế thừa lớp `NhanVien` và bổ sung thêm thuộc tính phụ cấp (`phu_cap`). Viết phương thức tính tổng thu nhập (lương cơ bản + phụ cấp). Nhập thông tin và in ra tổng thu nhập của 1 quản lý.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC12

Thời gian làm bài: 90 phút

Câu 1 (2 điểm): Nhập vào hai số nguyên dương a, b ($1 \leq a < b \leq 1000$). Viết chương trình:

- a) Liệt kê các số hoàn hảo trong đoạn $[a, b]$.
- b) Viết hàm kiểm tra một số có phải số thuận nghịch (palindrome) hay không.

Câu 2 (2 điểm): Viết chương trình nhập một chuỗi, thực hiện:

- a) Đếm số từ viết hoa toàn bộ trong chuỗi.
- b) Thay thế mọi khoảng trắng liên tiếp thành một dấu gạch dưới.
- c) In ra chuỗi sau thay thế.

Câu 3 (2 điểm): Nhập vào danh sách các số thực, thực hiện:

- a) Tính và in ra giá trị trung bình cộng.
- b) Tìm số nhỏ nhất và vị trí xuất hiện đầu tiên của nó.
- c) Tạo một list mới gồm các phần tử lớn hơn giá trị trung bình.

Câu 4 (2 điểm): Cho file `data.txt` chứa nhiều dòng, mỗi dòng một tên người. Viết chương trình:

- a) Đếm số dòng trùng nhau trong file.
- b) Ghi ra file `unique.txt` các tên xuất hiện đúng một lần.
- c) Thêm vào cuối file `data.txt` dòng: "Ho va ten: [HoTen cua ban]".

Câu 5 (2 điểm – OOP có kế thừa): Xây dựng lớp `NhanVien` với các thuộc tính: `ten`, `luong_cb` (lương cơ bản). Viết lớp con `NhanVienKyThuat` kế thừa, bổ sung thuộc tính `so_du_an` (số dự án hoàn thành), phương thức tính tổng lương theo công thức: tổng lương = lương cơ bản + $2.000.000 \times$ số dự án.

Nhập thông tin cho một nhân viên kỹ thuật, in ra tên, lương cơ bản, số dự án và tổng lương của người đó.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC13

Thời gian làm bài: 90 phút

Câu 1 (2.0 điểm): Viết chương trình nhập vào hai số nguyên dương x và y ($1 \leq x < y \leq 250$) và thực hiện các yêu cầu:

- Kiểm tra giá trị x, y nếu ngoài đoạn $[1; 250]$ hoặc $x \geq y$ thì yêu cầu nhập lại.
- Viết hàm in kết quả phép chia lấy phần nguyên và phần dư của y chia x .
- Viết hàm tính $S = \sum_{i=x}^y \frac{1}{i}$ và đếm có bao nhiêu số chia hết cho 5 trong đoạn $[x, y]$.

Câu 2 (2.0 điểm): Viết chương trình nhập một chuỗi ký tự từ bàn phím và thực hiện:

- Đếm số ký tự chữ cái có trong chuỗi.
- Tính tổng các chữ số trong chuỗi.
- Đổi tất cả ký tự đầu mỗi từ thành chữ hoa và đảo ngược thứ tự các từ trong chuỗi.

Câu 3 (2.0 điểm): Tạo một danh sách rỗng, viết chương trình xử lý danh sách với các chức năng sau:

- Nhập các số nguyên vào danh sách, dừng lại khi nhập 0. In danh sách đã nhập.
- Đếm số lượng số dương, số âm và số bằng 0 trong danh sách.
- Sắp xếp các số lẻ tăng dần ở đầu, số chẵn giảm dần ở cuối danh sách (giữ nguyên thứ tự gốc bên trong từng nhóm).

Câu 4 (2.0 điểm): Tạo file "input.txt" lưu 15 số nguyên (dòng đầu là số lượng, dòng sau là các số, mỗi số cách nhau bởi dấu cách).

- Đọc file và ghi ra file "sohoanhao.txt" các số hoàn hảo tìm được (nếu không có, ghi "Khong co so hoan hao").
- Tính tổng tất cả các số nguyên tố và ghi vào file "tong_nguyento.txt".
- Thêm dòng "Sinh vien: [Họ tên] - MSSV: [MSSV]" vào cuối file "input.txt".

Câu 5 (2.0 điểm): Tạo lớp cha `ThietBi` với thuộc tính: `Ten`, `Don gia`. Tạo lớp con `Laptop` kế thừa lớp `ThietBi`, bổ sung thuộc tính `RAM (GB)`, `trọng lượng (kg)`, phương thức in thông tin đầy đủ. Tạo 2 đối tượng: (HP, 27900000, 16, 1.9) và (Lenovo, 36800000, 32, 1.7) rồi in thông tin hai máy ra màn hình.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC14

Thời gian làm bài: 90 phút

Câu 1. Biến và kiểu dữ liệu chuỗi

- Nhập vào số nguyên y , kiểm tra và in ra y là số dương, số âm hay bằng 0. In lần lượt: **Duong**, **Am**, hoặc **Khong**.
- Viết chương trình nhập vào tên tỉnh và mã vùng của một tỉnh, sau đó in ra thông báo theo mẫu: **<Tỉnh> có mã vùng <Mã>**.
Ví dụ: Hà Nội có mã vùng 24.

Câu 2. Vòng lặp và câu lệnh điều kiện

- Nhập vào hai số nguyên x và y ($x \leq y$). Tính tổng tất cả các số âm trong đoạn từ x đến y (bao gồm cả x và y nếu phù hợp), in ra kết quả.
- Nhập vào một chuỗi ký tự và hai số nguyên x, y ($1 \leq x \leq y \leq n$), với n là độ dài chuỗi. Đếm số lượng ký tự nguyên âm (a, e, i, o, u, cả hoa và thường) xuất hiện trong chuỗi ở các vị trí từ x đến y (tính từ 1), in ra kết quả.

Câu 3. List – Đếm, chỉ mục và chèn phần tử

Cho danh sách ban đầu:

```
lst = [100, "apple", 3.14, "banana", 200, None]
```

- Viết chương trình tạo danh sách mới chỉ chứa các phần tử là số (int hoặc float) từ **lst**. In ra danh sách mới đó.
- Thực hiện các tác vụ sau:
 - Đếm số lần xuất hiện của phần tử 200 trong **lst** bằng phương thức **count()**. In ra kết quả.
 - Tìm chỉ mục (index) đầu tiên của phần tử "banana" trong **lst** bằng phương thức **index()**. In ra kết quả.
 - Chèn phần tử "grape" vào vị trí thứ 2 (index = 2) trong **lst** bằng phương thức **insert()**. In ra danh sách sau khi chèn.

Câu 4. Tuple – Khởi tạo và tính bất biến

- Viết chương trình khởi tạo (tạo) một tuple trống tên **empty_tuple**, một tuple chỉ có một phần tử số nguyên 5 (**single_tuple**), và một tuple chứa nhiều kiểu dữ liệu khác nhau (**mixed_tuple**) gồm 10, "apple", 3.14. In ra cả ba tuple này.
- Cho tuple **my_tuple = (1, 2, 3)**. Viết chương trình thử thay đổi giá trị tại chỉ mục 1 thành 9, và in ra thông báo lỗi (nếu có) hoặc mô tả kết quả.

Câu 5. Dictionary – Tính tổng, tìm max và cập nhật

- a) Cho dict `prices = {"apple": 20, "banana": 10, "orange": 15}`. Viết hàm `total_price(d)` trả về tổng giá trị các mặt hàng. Gọi hàm và in ra kết quả.
- b) Viết hàm `find_max_price(d)` trả về tên mặt hàng có giá cao nhất trong dict trên. Gọi hàm và in ra kết quả.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC15

Thời gian làm bài: 90 phút

Câu 1. Hàm biến đổi và phân tích chuỗi

- a) Viết hàm `camel_case(s)` nhận vào một chuỗi các từ cách nhau bởi dấu cách, trả về chuỗi dạng camelCase (từ đầu thường, các từ sau viết hoa chữ cái đầu, không chứa khoảng trắng). Gọi hàm với chuỗi "hello world python" và in ra kết quả.
- b) Viết hàm `longest_word(s)` nhận vào một chuỗi, trả về từ dài nhất trong chuỗi đó. Nếu có nhiều từ dài nhất, trả về từ xuất hiện đầu tiên. Gọi hàm với chuỗi "I am learning Python programming" và in ra kết quả.

Câu 2. Hàm thao tác hợp, giao, hiệu trên set

- a) Viết hàm `operations(s1, s2)` nhận vào hai tập hợp số nguyên, trả về tuple gồm (hợp, giao, hiệu) của hai tập hợp. In kết quả với $s1 = 1, 2, 3, 4$, $s2 = 3, 4, 5, 6$.
- b) Viết hàm `unique(s)` nhận vào một chuỗi, trả về tập hợp các ký tự khác nhau có trong chuỗi đó. In kết quả với chuỗi "banana".

Câu 3. Hàm thao tác và lọc tuple

- a) Viết hàm `odd_elements(tpl)` nhận vào một tuple số nguyên, trả về tuple mới chỉ chứa các phần tử lẻ của `tpl`. In ra kết quả với tuple (2, 3, 5, 8, 9, 12, 13).
- b) Viết hàm `tuple_stats(tpl)` nhận vào một tuple số thực, trả về tuple (trung bình cộng, tổng các phần tử). In ra kết quả với tuple (2.5, 3.5, 7.0, 6.0).

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC16

Thời gian làm bài: 90 phút

Câu 1. Hàm xử lý và kiểm tra tính chất trên chuỗi

- a) Viết hàm `reverse(s)` nhận vào một chuỗi, trả về chuỗi với thứ tự các từ bị đảo ngược (giữa các từ vẫn giữ nguyên). Gọi hàm với chuỗi "I love Python" và in ra kết quả.
- b) Viết hàm `palindrome(s)` nhận vào một chuỗi, trả về True nếu chuỗi là palindrome (đối xứng, không phân biệt hoa thường và bỏ qua khoảng trắng), ngược lại trả về False. Gọi hàm với chuỗi "A man a plan a canal Panama" và in ra kết quả.

Câu 2. Hàm kiểm tra và biến đổi trên set

- a) Viết hàm `subset(s1, s2)` kiểm tra s1 có phải là tập con của s2 hay không. Gọi hàm với `s1 = 1, 2`, `s2 = 1, 2, 3` và in ra kết quả.
- b) Viết hàm `squared(s)` nhận vào một set các số nguyên, trả về set mới gồm bình phương các phần tử trong s. In kết quả với `s = 1, 3, 4`.

Câu 3. Hàm xử lý tuple lồng nhau và tìm kiếm

- a) Viết hàm `flatten_tuple(tp1)` nhận vào một tuple lồng các tuple con, trả về một tuple mới chứa tất cả các phần tử (làm phẳng tuple). In ra kết quả với tuple `((1, 2), (3, 4, 5), (6,))`.
- b) Viết hàm `first_str(tp1)` nhận vào một tuple bất kỳ, trả về phần tử đầu tiên là kiểu chuỗi (str), nếu không có trả về None. In ra kết quả với tuple `(5, 2.5, "abc", True, "xyz")`.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC17

Thời gian làm bài: 90 phút

Câu 1. List – Tìm phần tử lớn nhất

Cho danh sách ban đầu:

```
lst = [100, "apple", 3.14, "banana", 200, None]
```

- Viết hàm `max_num(lst)` nhận vào một list, trả về số lớn nhất trong list (chỉ xét các phần tử là số). In ra kết quả với list trên.
- Viết chương trình tạo list mới `lst_str` gồm các phần tử của `lst` đã được chuyển thành chuỗi (str), rồi in ra list mới đó.

Câu 2. Vòng lặp – Phân tích chuỗi số

- Nhập vào một chuỗi số (gồm toàn chữ số, ví dụ: "1234567"). Tính tổng các chữ số lẻ trong chuỗi và in ra kết quả.
- Đếm số lượng cặp hai chữ số liên tiếp trong chuỗi có tổng bằng 10. In ra kết quả.

Câu 4. Tuple – Unpacking và tuple lồng nhau

- Cho tuple `person = ("Binh", 23, "Hanoi")`. Viết chương trình unpack (gán) các giá trị của tuple này cho ba biến `ten`, `tuoi`, `que` rồi in ra theo định dạng: Binh - 23 - Hanoi.
- Cho tuple lồng nhau `matrix = ((1,2,3), (4,5,6), (7,8,9))`. Viết chương trình in ra tổng tất cả các phần tử trong `matrix`.

ĐỀ KIỂM TRA PYTHON NÂNG CAO - NC18

Chủ đề: Hàm nâng cao · List/Dict comprehension · File I/O · Xử lý ngoại lệ

Thời gian làm bài: 90 phút

Câu 1. Hàm nâng cao và tham số linh hoạt

- a) Viết hàm `thong_ke(*args)` nhận vào một số lượng bất kỳ các số nguyên. Hàm trả về một **dictionary** gồm 4 key: `"min"`, `"max"`, `"sum"`, `"avg"` (trung bình cộng, làm tròn 2 chữ số thập phân). Nếu không truyền tham số nào, trả về dict rỗng. Gọi hàm với ít nhất 2 ví dụ và in kết quả.
- b) Viết hàm `loc_du_lieu(data, **kwargs)` trong đó `data` là một list các dictionary (mỗi dict đại diện một sinh viên có các key `"ten"`, `"tuoi"`, `"diem"`). Các `**kwargs` là điều kiện lọc, ví dụ `tuoi=20` hoặc `diem=8.5`. Hàm trả về list các sinh viên thỏa mãn **tất cả** điều kiện. In ra kết quả.
Ví dụ: `loc_du_lieu(students, tuoi=20, diem=8.5)` → trả về các SV có tuổi=20 VÀ điểm=8.5.

Câu 2. List comprehension và Dict comprehension

- a) Cho danh sách `words = ["Hello", "World", "Python", "AI", "Data", "Go"]`. Dùng **list comprehension** để tạo danh sách mới chỉ chứa các từ có độ dài > 3 ký tự, và chuyển tất cả sang **chữ thường**. In ra kết quả.
- b) Cho danh sách `scores = [45, 78, 92, 55, 88, 34, 67, 91, 72, 50]`. Dùng **dict comprehension** để tạo một dictionary với key là điểm số, value là xếp loại theo quy tắc: $\geq 80 \rightarrow$ "Gioi", $\geq 60 \rightarrow$ "Kha", $\geq 50 \rightarrow$ "TB", $< 50 \rightarrow$ "Yeu". In ra dictionary.
Gợi ý: Có thể dùng biểu thức điều kiện lồng nhau trong comprehension.

Câu 3. Xử lý ngoại lệ (Exception Handling)

- a) Viết hàm `chia_an_toan(a, b)` thực hiện phép chia a/b . Xử lý các ngoại lệ: `ZeroDivisionError` (in "Loi: chia cho 0"), `TypeError` (in "Loi: kieu du lieu khong hop le"). Trong mọi trường hợp, in "Ket thuc phep chia" bằng khối `finally`. Gọi hàm với 3 trường hợp: chia bình thường, chia cho 0, và truyền chuỗi.
- b) Viết hàm `doc_diem(filename)` đọc file có tên `filename`, mỗi dòng chứa một số thực (điểm). Hàm trả về list các điểm hợp lệ (bỏ qua dòng trống hoặc dòng không phải số). Xử lý `FileNotFoundError` nếu file không tồn tại (in thông báo lỗi phù hợp). Xử lý `ValueError` cho từng dòng không chuyển được sang float. Cuối cùng in tổng số dòng hợp lệ và điểm trung bình.

Câu 4. Đọc và ghi File

- a) Viết chương trình tạo file `sinhvien.txt` với nội dung sau (mỗi dòng gồm Tên, Tuổi, Điểm cách nhau bởi dấu phẩy):

```
Minh,20,8.5
Lan,21,9.0
Hung,19,6.5
Mai,20,7.8
Tuan,22,5.5
```

Sau đó đọc file, phân tách dữ liệu, và in ra danh sách sinh viên có điểm ≥ 7.0 theo định dạng: `Ten: <Ten> Diem: <Diem>|`.

- b) Viết chương trình đọc file `sinhvien.txt` ở câu a, tính điểm trung bình của tất cả sinh viên, tìm sinh viên có điểm cao nhất, rồi ghi kết quả vào file mới `ketqua.txt` theo mẫu:

```
Diem trung binh lop: 7.46
Sinh vien xuat sac nhat: Lan (9.0 diem)
```

Câu 5. Hàm đệ quy và Lambda

- a) Viết hàm đệ quy `dem_chu_so(n)` nhận vào một số nguyên dương `n` và trả về số lượng chữ số của `n`. **Không** được dùng `str()` hay `len()`. Gọi hàm và in kết quả với `n = 123456` và `n = 7`.

Gợi ý: Cơ sở: $n < 10 \rightarrow 1$ chữ số. Đệ quy: $1 + \text{dem_chu_so}(n // 10)$.

- b) Cho danh sách `students` gồm các tuple `(ten, diem)`: `[("Minh", 8.5), ("Lan", 9.0), ("Hung", 7.5), ("Mai", 6.5), ("Tuan", 5.5)]`. Dùng hàm `sorted()` kết hợp `lambda` để sắp xếp danh sách theo điểm giảm dần. In ra danh sách đã sắp xếp.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC19

Chủ đề: OOP cơ bản · Kế thừa · Module · Xử lý dữ liệu có cấu trúc

Thời gian làm bài: 90 phút

Câu 1. Lập trình hướng đối tượng – Class cơ bản

a) Viết class `SinhVien` với các thuộc tính: `ten` (str), `mssv` (str), `diem` (list các float). Viết các method:

- 1) `diem_tb()` – trả về điểm trung bình (làm tròn 2 chữ số thập phân).
- 2) `xep_loai()` – trả về xếp loại dựa trên điểm TB: $\geq 8.5 \rightarrow \text{"Gioi"}; \geq 7.0 \rightarrow \text{"Kha"}; \geq 5.0 \rightarrow \text{"Trung binh"}; < 5.0 \rightarrow \text{"Yeu"}.$
- 3) `__str__()` – trả về chuỗi dạng `"SV <ten> (<mssv>) - <xep loai>".`

Tạo 3 đối tượng `SinhVien`, gọi tất cả method và in kết quả.

b) Viết class `LopHoc` chứa danh sách các `SinhVien`. Viết các method:

- 1) `them_sv(sv)` – thêm sinh viên vào lớp.
- 2) `diem_tb_lop()` – trả về điểm trung bình toàn lớp.
- 3) `sv_xuat_sac()` – trả về sinh viên có điểm TB cao nhất.

Tạo một lớp học có 3 sinh viên ở câu a, gọi các method và in kết quả.

Câu 2. Kế thừa (Inheritance) và đa hình

a) Viết class cha `HinhHoc` có method `dien_tich()` trả về 0 và `__str__()` trả về tên hình. Viết 2 class con:

- 1) `HinhTron(ban_kinh)` – override `dien_tich()` trả về $\pi \times r^2$ (dùng `math.pi`), `__str__()` trả về `"Hinh tron (r=<ban kinh>)".`
- 2) `HinhChuNhat(dai, rong)` – override `dien_tich()` trả về `dai × rong`, `__str__()` trả về `"HCN (<dai>x<rong>)".`

b) Viết hàm `in_thong_tin(hinh)` nhận một đối tượng bất kỳ kế thừa từ `HinhHoc`, in ra tên hình và diện tích (làm tròn 2 chữ số). Tạo list gồm 2 hình tròn và 2 hình chữ nhật, duyệt qua list và gọi `in_thong_tin()` cho mỗi hình. Giải thích trong comment tại sao đây là ví dụ của đa hình (polymorphism).

Câu 3. Module tự tạo và import

a) Tạo module `math_utils.py` chứa 3 hàm:

- 1) `giai_thua(n)` – trả về $n!$ bằng đệ quy. Raise `ValueError` nếu $n < 0$.

2) `la_so_nguyen_to(n)` – trả về `True` nếu n là số nguyên tố.

3) `fibonacci(n)` – trả về list n số Fibonacci đầu tiên.

b) Tạo file `main.py`, import module `math_utils` và thực hiện:

1) In giai thừa của 5, 7, 10.

2) In tất cả số nguyên tố trong đoạn $[2, 50]$.

3) In 15 số Fibonacci đầu tiên.

Câu 4. Xử lý dữ liệu có cấu trúc – JSON

a) Cho dữ liệu sau (dạng list of dict):

```
products = [
    {"ten": "Laptop", "gia": 15000000, "sl": 5},
    {"ten": "Chuot", "gia": 350000, "sl": 50},
    {"ten": "Ban phim", "gia": 800000, "sl": 30},
    {"ten": "Man hinh", "gia": 5500000, "sl": 10},
    {"ten": "Tai nghe", "gia": 1200000, "sl": 25}
]
```

Viết chương trình: (1) Tính tổng giá trị tồn kho (giá $\times$ số lượng) của tất cả sản phẩm.

(2) Tìm sản phẩm có giá cao nhất. (3) Lọc các sản phẩm có giá trị tồn kho $> 10.000.000$ VND. In kết quả mỗi phần.

b) Ghi danh sách `products` ở trên vào file `products.json` bằng module `json` (đảm bảo tiếng Việt hiển thị đúng). Sau đó đọc lại file, thêm sản phẩm mới `{"ten": "USB", "gia": 150000, "sl": 100}` vào danh sách, ghi lại file. In ra nội dung file cuối cùng.

Câu 5. Tổng hợp – Quản lý cửa hàng mini

a) Viết class `SanPham` với thuộc tính `ten`, `gia`, `so_luong`. Viết method `gia_tri()` trả về `gia  $\times$  so_luong` và `__str__()` trả về chuỗi thông tin sản phẩm.

b) Viết class `CuaHang` chứa danh sách `SanPham`. Viết các method:

1) `them_sp(sp)` – thêm sản phẩm. Nếu trùng tên, cộng dồn số lượng.

2) `tong_gia_tri()` – tổng giá trị tồn kho.

3) `tim_sp(ten)` – tìm sản phẩm theo tên, trả về đối tượng hoặc `None`.

4) `xuat_bao_cao(filename)` – ghi báo cáo ra file text với danh sách sản phẩm và tổng giá trị.

Tạo cửa hàng với ít nhất 4 sản phẩm, gọi tất cả method và in kết quả.

ĐỀ KIỂM TRA PYTHON NÂNG CAO – NC20

Chủ đề: Decorator · Generator · Xử lý text nâng cao · Thuật toán · Thiết kế hệ thống

Thời gian làm bài: 90 phút

Câu 1. Decorator và hàm bậc cao

- a) Viết decorator `@dem_goi` đếm số lần một hàm được gọi. Mỗi lần hàm được gọi, in ra "`<ten ham> da duoc goi <n> lan`". Áp dụng decorator cho hàm `chao(ten)` và gọi 3 lần liên tiếp.
Gợi ý: Dùng thuộc tính của hàm wrapper, ví dụ `wrapper.count = 0`.
- b) Viết decorator `@validate_positive` kiểm tra tất cả tham số truyền vào hàm đều là số dương. Nếu có tham số ≤ 0 , raise `ValueError` với thông báo "`Tat ca tham so phai la so duong`". Áp dụng cho hàm `tinh_dien_tich(dai, rong)` và test với cả input hợp lệ và không hợp lệ.

Câu 2. Generator và Iterator

- a) Viết hàm generator `so_nguyen_to_gen(limit)` sinh ra tất cả các số nguyên tố nhỏ hơn `limit` bằng từ khóa `yield`. Dùng generator này để in tất cả số nguyên tố nhỏ hơn 100.
- b) Viết class `DayFibonacci` implement **iterator protocol** (có method `__iter__()` và `__next__()`). Constructor nhận tham số `n` (số phần tử cần sinh). Khi dùng trong vòng lặp `for`, class sinh ra `n` số Fibonacci đầu tiên. Raise `StopIteration` khi đã sinh đủ.
Ví dụ: `for x in DayFibonacci(8): print(x)` → 0, 1, 1, 2, 3, 5, 8, 13.

Câu 3. Xử lý chuỗi và text nâng cao

- a) Viết hàm `dem_tan_suat(text)` nhận một đoạn văn bản (chuỗi), trả về dictionary với key là từng từ (chuyển thường, loại bỏ dấu câu), value là số lần xuất hiện. Sắp xếp dictionary theo giá trị giảm dần. Test với đoạn văn bản ít nhất 3 câu.
Gợi ý: Dùng `str.translate()` hoặc `re.sub()` để loại bỏ dấu câu.
- b) Viết hàm `ma_hoa_caesar(text, shift)` mã hóa chuỗi bằng Caesar cipher: dịch mỗi ký tự chữ cái đi `shift` vị trí trong bảng chữ cái (chỉ xử lý a-z và A-Z, giữ nguyên ký tự khác). Viết hàm `giai_ma_caesar(text, shift)` để giải mã. Test mã hóa rồi giải mã chuỗi "Python la ngon ngu lap trinh so 1!" với `shift=3`.

Câu 4. Thuật toán – Sắp xếp và tìm kiếm

- a) Viết hàm `bubble_sort(lst)` sắp xếp danh sách số nguyên tăng dần bằng thuật toán **Bubble Sort**. Không được dùng `sort()` hay `sorted()`. Hàm trả về danh sách đã sắp xếp. Test với [64, 34, 25, 12, 22, 11, 90].

- b) Viết hàm `binary_search(lst, target)` tìm kiếm phần tử `target` trong danh sách **đã sắp xếp** `lst` bằng thuật toán **tìm kiếm nhị phân**. Trả về chỉ mục (index) nếu tìm thấy, trả về `-1` nếu không. **Không** được dùng `index()` hay toán tử `in`. Dùng kết quả Bubble Sort ở câu a để test.

Câu 5. Thiết kế hệ thống – Quản lý thư viện sách

Thiết kế hệ thống quản lý thư viện sách mini với các class sau:

- a) Class `Sach` với thuộc tính: `tua_de`, `tac_gia`, `nam_xb` (int), `trang_thai` ("Co san" hoặc "Dang muon"). Viết method `__str__()`, `muon()` (chuyển trạng thái), `tra()` (chuyển trạng thái). Raise lỗi nếu mượn sách đang được mượn.
- b) Class `ThuVien` chứa danh sách `Sach`. Viết các method:
- 1) `them_sach(sach)` – thêm sách vào thư viện.
 - 2) `tim_theo_tac_gia(tac_gia)` – trả về list sách của tác giả.
 - 3) `sach_co_san()` – trả về list sách có trạng thái "Co san".
 - 4) `thong_ke()` – in tổng số sách, số sách có sẵn, số sách đang mượn.
 - 5) `xuat_file(filename)` – ghi danh sách sách ra file text.

Tạo thư viện với ít nhất 5 cuốn sách, thực hiện mượn 2 cuốn, trả 1 cuốn, tìm kiếm theo tác giả, in thống kê, và xuất file. In kết quả từng bước.

— Hết — Được sử dụng tài liệu cá nhân — TVS